STUDENT ERP SYSTEM

Project Report

A React + TypeScript Web-Based School Management Application

Submitted in partial fulfillment of the requirements
for the award of the degree of Bachelor of Computer Applications (BCA)

Prepared by:

Pranali Bagilgekar

github.com/Pranali027-blip/student-management-system

1. Project Overview

The Student ERP System, Edu-Center, is a browser-based school management dashboard developed using React 18, TypeScript, and Vite. It is designed to support small to mid-sized educational institutions in managing day-to-day academic and administrative operations without the need for backend infrastructure, enabling fast and accessible deployment across modern web browsers.

The application comprises five core modules—Dashboard, Students, Attendance, Exams, and Notifications—implemented using a modular, component-based architecture with seamless navigation via React Router v6. Developed as a portfolio project, it highlights proficiency in TypeScript, custom CSS, and scalable front-end design practices without reliance on external UI or charting libraries.

Project Title	Student ERP System (Edu-Center)
Subtitle	Web-Based School Management Application
Developed By	Pranali Bagilgekar
Tech Stack	React 18, TypeScript, Vite, React Router v6, Custom CSS
Total Modules	5 — Dashboard, Students, Attendance, Exams, Notifications
Application Type	Single Page Application (SPA) — Front-End Only
Lines of Code	~2,000+ (TSX + CSS)
GitHub	github.com/Pranali027-blip/student-management-system
Status	Completed — v1.0.0 | April 2025

2. Objectives

The following objectives were set before development began:

- Build a complete, production-quality school management dashboard entirely on the front-end using React 18 and TypeScript
- Implement full CRUD operations (Create, Read, Update, Delete) for student records and exam schedules
- Build interactive data visualisations — including area charts, donut charts, and bar charts — without using any third-party charting library
- Implement real-time search and filter functionality across all modules so users can instantly find what they need
- Design a consistent, professional UI with a deep purple sidebar, gradient accents, and responsive card layouts — using only custom CSS, no UI framework
- Apply strict TypeScript throughout the codebase — typed interfaces for all data models, union types for status fields, and no use of 'any'
- Demonstrate readiness for a junior-to-mid-level front-end developer role through independent full-project delivery

3. Technology Stack

Technology	Version	Why It Was Used
React	v18+	Component-based UI framework; ideal for building a multi-page SPA with reusable modules
TypeScript	v5+	Adds static type-checking; prevents bugs at compile time; improves code readability
Vite	v5+	Extremely fast build tool and dev server; HMR under 300ms; replaces deprecated CRA
React Router v6	v6	Handles client-side routing across 5 pages with no full-page reloads
Custom CSS	—	All styling hand-coded; no Tailwind or MUI; demonstrates pure CSS and layout skills
React Hooks	Built-in	useState and useEffect manage all application state and side effects
Git / GitHub	—	Version control and public portfolio hosting

3.1 Why No Component Library?

A deliberate decision was made not to use any component library such as Material UI, Ant Design, or Tailwind CSS. Every card, modal, table, badge, sidebar, and button in this project is hand-coded in CSS. This choice was made to demonstrate that the developer can build production-quality UI from scratch — a skill that matters significantly in technical interviews and for employers who want developers who understand CSS fundamentals deeply.

3.2 Why Vite Over Create React App?

Create React App (CRA) is no longer actively maintained by the React team and comes with heavy overhead. Vite's dev server cold-starts in under 300 milliseconds and its Hot Module Replacement (HMR) is near-instantaneous. For a project of this scale, switching to Vite noticeably improved iteration speed during development and produces smaller, faster production bundles via its Rollup-based build pipeline.

4. Application Architecture

4.1 Folder Structure

The project follows the standard Vite + React folder structure, with a flat pages directory and scoped CSS files per module:

```
src/
  App.tsx                  —  Route definitions (5 routes)
  main.tsx                 —  Entry point, BrowserRouter wrapper
  pages/
    Dashboard.tsx          —  KPI cards, charts, calendar, events, finance
    Students.tsx           —  Student CRUD, search, filter, pagination
    Attendance.tsx         —  Daily attendance marking, charts, analytics
    Exams.tsx              —  Exam scheduling, status filter, delete modal
    Notifications.tsx      —  Notification centre, type filter, mark read
  Dashboard.css            —  Shared sidebar, topbar, layout styles
  Students.css             —  Student module styles (stu- prefix)
  Attendance.css           —  Attendance module styles (att- prefix)
  Exams.css                —  Exam module styles (exam- prefix)
  Notifications.css        —  Notification styles (notif- prefix)
```

4.2 Routing

Client-side routing is implemented using React Router v6 with BrowserRouter. Five routes are registered in App.tsx: / (Dashboard), /students, /attendance, /exam, and /notifications. Every page's sidebar uses the useNavigate() hook to navigate programmatically, so clicking any sidebar item updates the URL and renders the correct module without refreshing the browser. This gives users a smooth, app-like experience.

4.3 State Management

All state in this application is managed at the component level using React's useState hook. There is no global state manager (Redux, Zustand, or Context API) — a deliberate choice for a project of this scale. Each page component owns its own state: the student list, form data, search query, current page number for pagination, modal open/close state, and filter selections. This keeps each module independent and easy to reason about. The architecture is designed so that a global state layer or React Query (TanStack Query) can be added later when a back-end is integrated.

4.4 CSS Architecture

Each page has a dedicated CSS file with a page-specific class prefix — stu- for Students, att- for Attendance, exam- for Exams, and notif- for Notifications. This prevents class name collisions across modules without needing CSS Modules or styled-components. The shared Dashboard.css file contains the sidebar, topbar, KPI card layout, and shared button and badge styles used across all five pages. All CSS variables for the purple gradient (#1E3A5F to #5B35D5) and accent colour (#6366F1) are defined in this shared file.

5. Module-by-Module Explanation

5.1 Dashboard Module

The Dashboard is the home screen of the application, accessible at the root route (/). It serves as the central command view for school administrators, displaying school-wide statistics and trends at a glance. The page is divided into three visual sections: KPI stat cards at the top, a main content area with charts and calendar, and a right sidebar with upcoming events and finance data.

KPI Stat Cards

Four stat cards are displayed prominently at the top of the dashboard. As visible in the screenshots, these show: Total Students (6,825), Total Teachers (654), Events (656), and Invoice Status (1,397). Each card includes a trend indicator — green for positive growth (e.g. '+0.5% than last month') and red for decline (e.g. '-3% than last month'). The cards use a white background with a coloured left-border accent system and a subtle hover lift animation using CSS transform: translateY(-2px).

School Performance Chart

Below the KPI cards sits a School Performance area chart that plots Students and Teachers trend lines across 10 weeks (Week 01 to Week 10). This chart is rendered entirely as inline SVG — no chart library is used. The two trend lines use smooth curves with filled gradient areas beneath them, giving a professional chart appearance. A dropdown allows toggling between 'This Month', making the chart feel dynamic. The Students line (purple) and Teachers line (green) make it easy to visually compare growth patterns at a glance.

School Event Calendar

A fully functional month-view calendar is rendered using pure JavaScript date calculations — no date picker library. The calendar correctly calculates the first weekday of the month and renders the full grid from Monday to Sunday. Today's date (April 22) is highlighted in a purple filled circle. Days with scheduled events are marked with small coloured dots beneath the date number. A month and year dropdown pair allow navigation between months.

School Finance Panel

The right side of the Dashboard shows a School Finance section with Monthly/Weekly toggle buttons. It displays Income ($4,69,244) and Expense ($33,456) as summary figures with icon indicators. A smaller SVG line chart beneath plots the income and expense trend lines across the week (Sun through Sat). An Upcoming Events panel on the right lists future school events with date badges, event names, ticket sales progress, and colour-coded event-type dots.

Upcoming Exams Alert

A highlighted call-to-action card in the top-right of the Dashboard alerts the user that 'Upcoming Exams Are Near!' and provides a 'View Exam Schedule' button that navigates directly to the Exam module. This is a

practical UX touch that reduces clicks for the most time-sensitive information a teacher or admin would need.

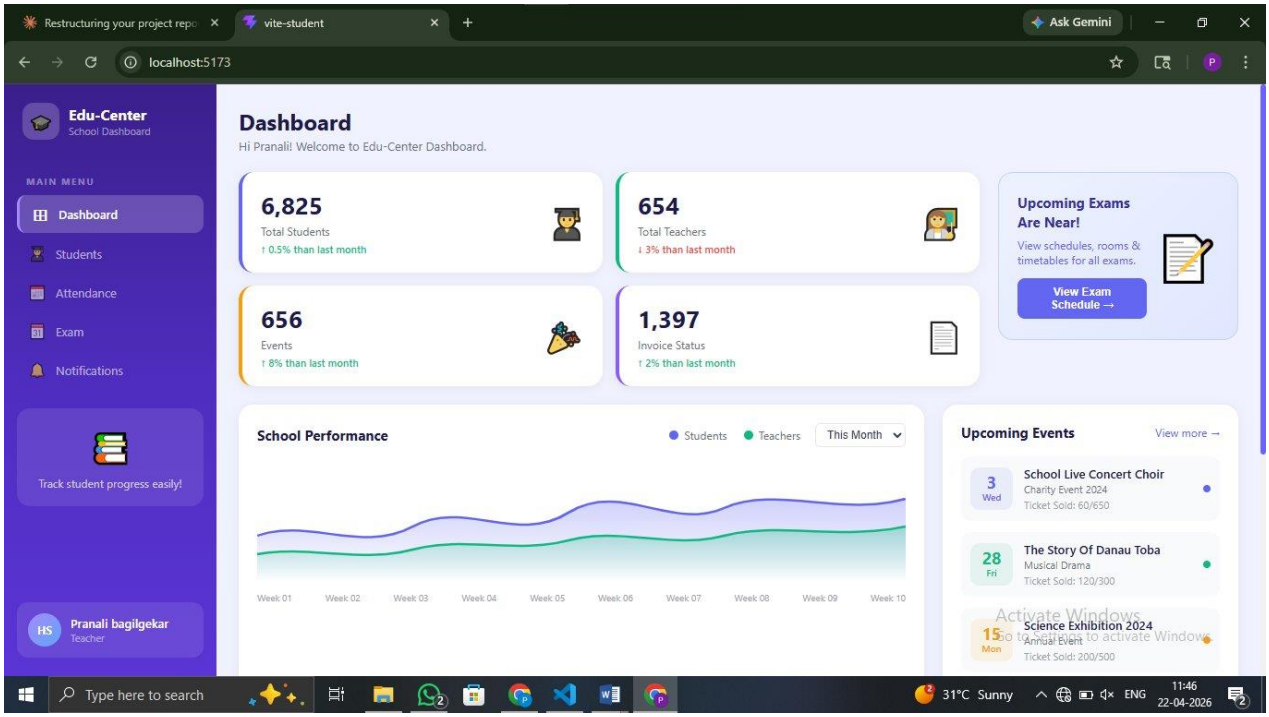

Figure 1 — Dashboard: KPI Cards, Performance Chart, and Upcoming Events

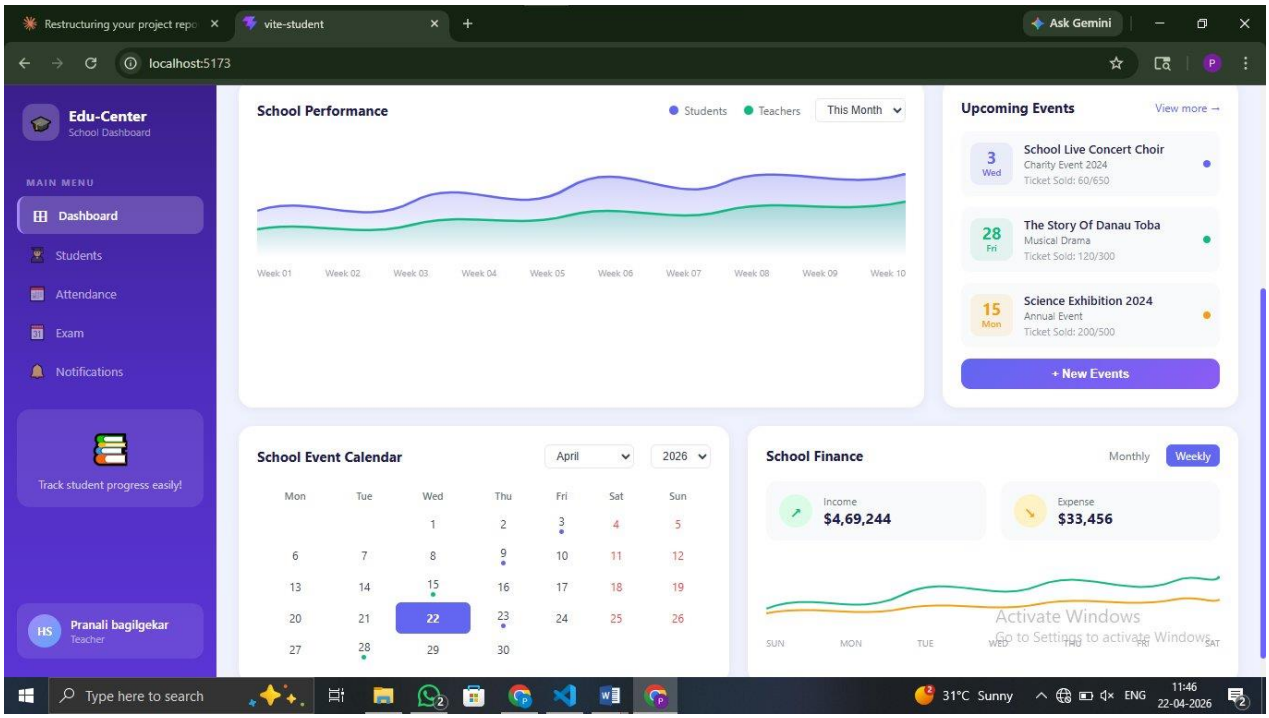

Figure 2 — Dashboard: School Event Calendar and Finance Panel

5.2 Student Management Module

The Students module, accessible at /students, is the core data management section of the system. It provides a complete interface for managing enrolled student records — adding new students, viewing all records in a paginated table, editing existing data inline via a modal, and deleting records.

Summary Statistics

At the top of the Students page, four stat cards show the current state of the student database: Total Students (12), Active Students (9), Inactive Students (3), and Average Score (83%). Each card includes a month-over-month trend indicator showing how these numbers have changed, providing administrators with instant context without having to count records manually.

All Students Table

The main content area contains the All Students table. Each row shows: a sequential row number, the student's auto-generated avatar initial (a coloured circle with the first letters of their name — e.g. 'AS' for Aarav Sharma), their full name, grade (9th–12th), subject (Science, Mathematics, Commerce, Arts), a colour-coded score progress bar with the numerical score, an Active/Inactive status badge, and three action icons — View (eye icon), Edit (pencil icon), and Delete (trash icon).

Real-Time Search and Grade Filter

A search bar above the table allows administrators to type any part of a student's name and the table filters instantly on every keystroke using JavaScript's .filter() and .toLowerCase() methods. A dropdown beside the search bar allows filtering by grade (All Grades, 9th, 10th, 11th, 12th). Both filters can work simultaneously — searching for 'Sharma' while filtering for '10th' would show only 10th-grade students whose name contains 'Sharma'.

Score Progress Bar

Each student's score is visualised as a horizontal progress bar in the Score column. The fill colour transitions dynamically from red (low scores) through orange and yellow to green (high scores) based on the score value. This gives examiners and teachers an immediate visual sense of student performance without reading individual numbers.

Pagination

The table displays 8 students per page. At the bottom of the table, pagination controls show 'Showing 1–8 of 12 students' with Previous and Next buttons and numbered page buttons (1, 2). The Previous button is disabled on the first page and the Next button is disabled on the last page. Pagination logic is implemented manually using JavaScript array slice — no external pagination library is used.

Top Performers Sidebar

On the right side of the Students page, a 'Top Performers This Month' panel ranks the top 4 students by score with gold, silver, and bronze medal icons. It shows: Sneha Kulkarni (95, 10th Arts), Pooja Nair (93, 11th

Mathematics), Aarav Sharma (92, 10th Science), and Ananya Desai (91, 9th Mathematics). Below this, a Subject Breakdown section shows a mini bar chart of student distribution across Science, Mathematics, Commerce, and Arts (3 students each).

Add / Edit Student Modal

Clicking the '+ Add Student' button in the top-right opens a modal overlay with a form containing fields for student name, grade, subject, score, and status. All fields include inline validation — if a required field is left empty on submit, an error message appears directly below that field (e.g. 'Name is required'). Editing a student pre-fills the modal with the existing data. On save, the record is instantly updated in the state array using .map(), and the table reflects the change immediately.

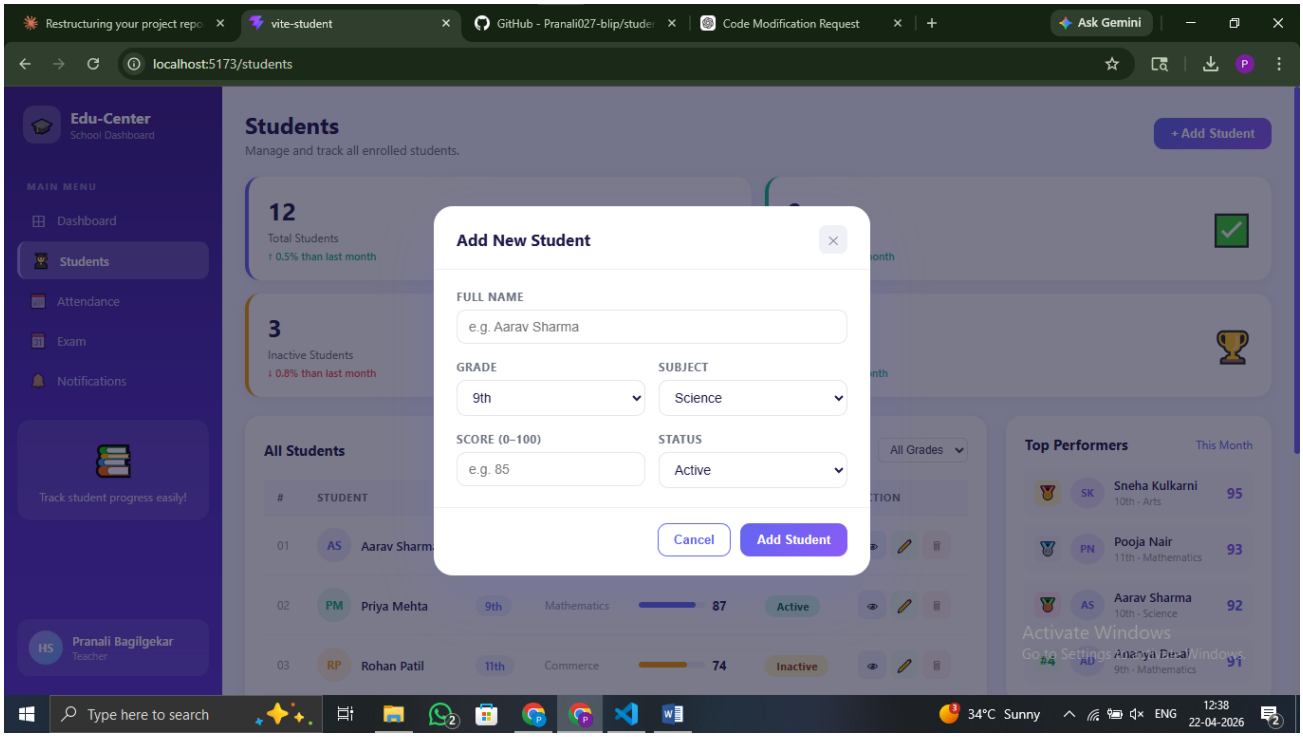

Figure 3 — Students Module: Stat Cards, Table, Search, and Top Performers

5.3 Attendance Tracking Module

The Attendance module at /attendance allows teachers to mark and track student attendance for each school day. It is one of the most visually rich modules in the application, featuring a live donut chart, a weekly bar chart, and a per-grade breakdown — all built without any charting library.

Daily Summary Cards

Four stat cards at the top of the page show today's attendance snapshot: Total Students (12), Present Today (7 — 58% attendance), Absent Today (3 — 25% of class), and Late Arrivals (2 — 17% of class). The current date is displayed in the top-right as a formatted date badge (e.g. 'Thursday, 18 Jan 2024'), making it clear which day's data is being viewed.

Today's Overview — SVG Donut Chart

The 'Today's Overview' panel shows a circular donut chart displaying the attendance percentage breakdown. The chart is rendered in pure SVG using the stroke-dasharray technique: each segment's arc length is calculated from the percentage (Present 58%, Late 17%, Absent 25%) and applied as a stroke dasharray on a circle element. The chart animates smoothly on page load using CSS transitions. A legend below the chart shows the colour coding: green for Present (7 students), orange for Late (2 students), and red for Absent.

Weekly Attendance Bar Chart

The 'Weekly Attendance' section shows a grouped bar chart for the current week (Mon–Fri). For each day, three stacked mini bars represent Present (green), Late (orange), and Absent (red) counts. These bars are built using flex-column div elements with their height set as an inline percentage style calculated from the attendance data. CSS transitions animate the bars on render. A colour legend is shown below the chart.

By Grade Breakdown

The right panel shows attendance percentages broken down by grade: Grade 9th (67%, blue bar), Grade 10th (100%, full blue bar), Grade 11th (33%, partial bar). This gives teachers an instant view of which grades have attendance problems without having to count individual records.

Attendance Register Table

The lower section of the Attendance page shows the 'Attendance Register — Today' table. Each row displays the student's avatar, name, grade, subject, time-in (e.g. 08:45 AM), a colour-coded status badge (Present/Late/Absent), a Quick Mark section with three buttons (P in green, L in yellow, A in red), and an edit action. Clicking P, L, or A instantly updates that student's attendance status — only one button can be active at a time, implementing a radio-button pattern using CSS active state. A search bar and All/Present/Late/Absent filter dropdown sit above the table.

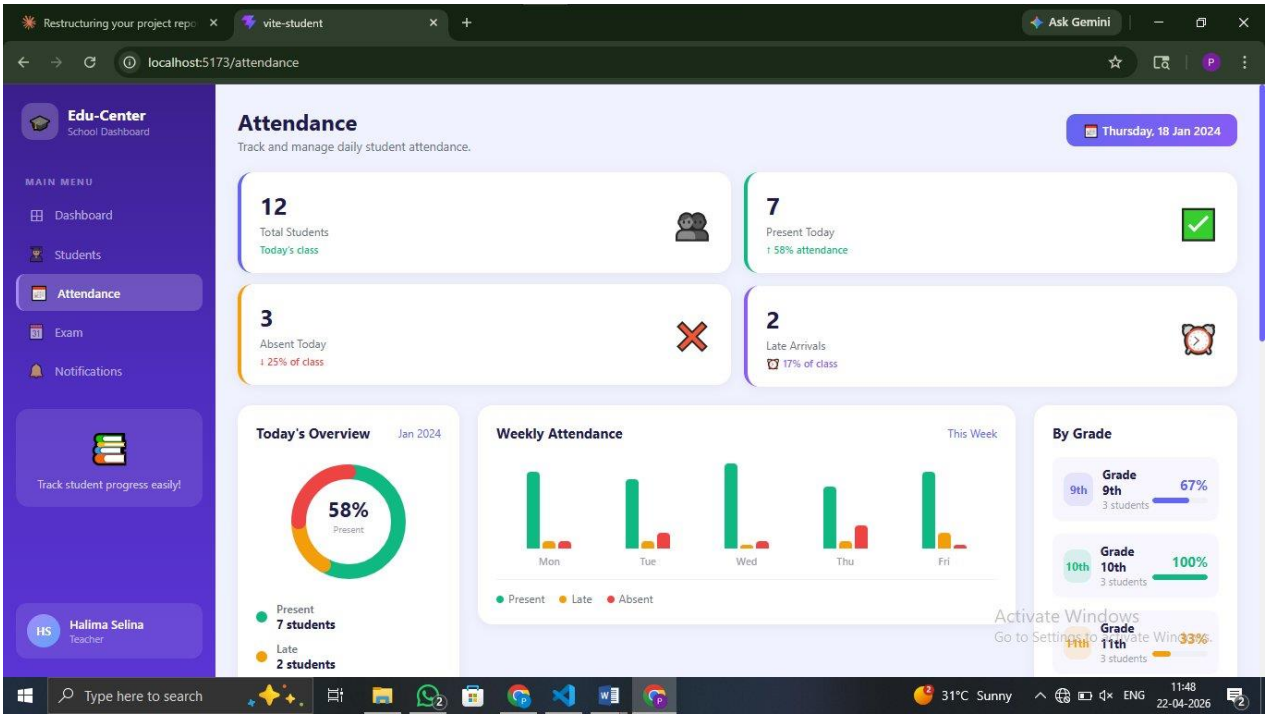

Figure 5 — Attendance Module: Stat Cards, Donut Chart, Weekly Bar Chart, and Grade Breakdown

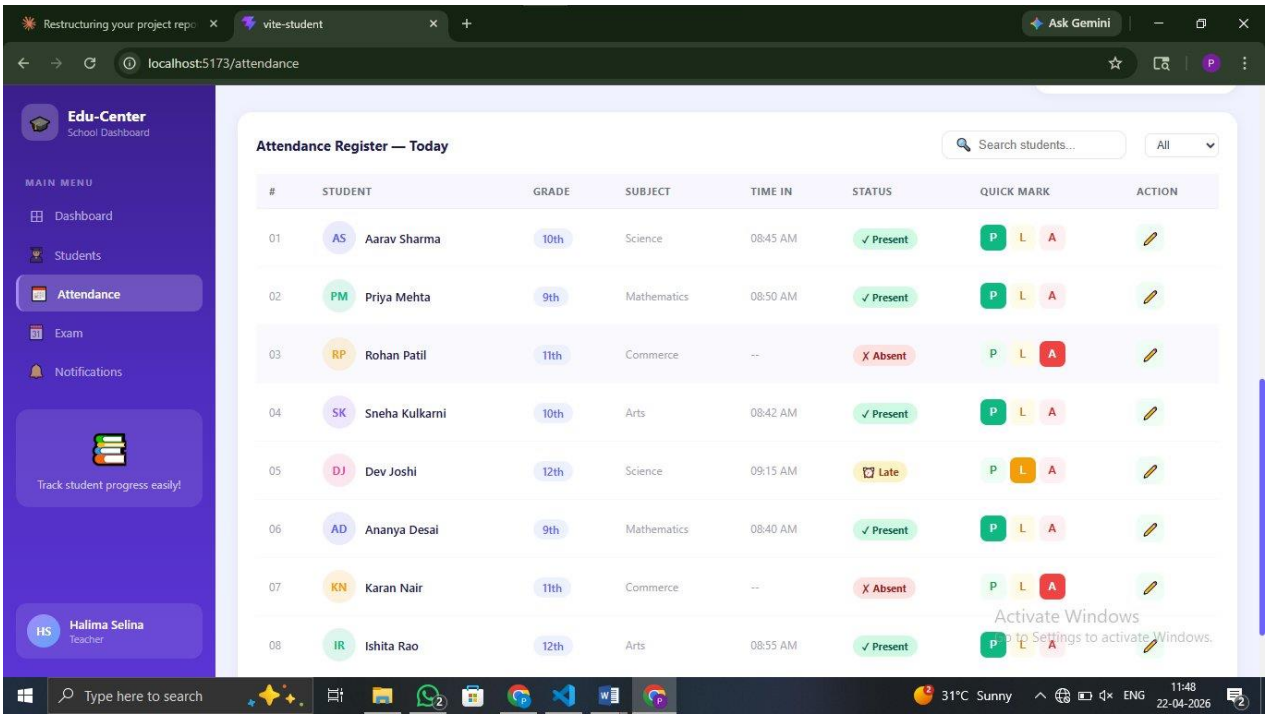

Figure 6 — Attendance Module: Daily Attendance Register with Quick Mark Buttons

5.4 Exam Scheduling Module

The Exam module at /exam allows administrators to schedule, track, and manage all school examinations. It provides a full view of the exam calendar with real-time status tracking and filter capabilities.

Exam Summary Cards

Four stat cards at the top show the current exam situation: Total Exams (6), Upcoming (3 — 'Scheduled ahead'), Ongoing (1 — 'Live now'), and Completed (2 — 'Done'). The Ongoing card has a red live indicator dot, making it immediately clear which exam is currently in progress. This is a practical feature for exam coordinators monitoring real-time exam sessions.

All Exams Table

The main table lists all scheduled exams with the following columns: serial number, Subject, Code (e.g. MATH-101, PHY-102, ENG-103), Type (Final, Mid-Term, Unit Test), Date, Time, Duration, Room, Students (number of students appearing), Status badge, and a Delete action. The exam code badges are styled in blue pill shapes, making them stand out for quick identification. Status badges are colour-coded: Upcoming in blue, Ongoing in red/orange, and Completed in green.

Search and Status Filter

A search bar allows filtering exams by subject name or exam code. Four filter buttons — All, Upcoming, Ongoing, Completed — allow instant filtering of the table by status. Clicking 'Upcoming' would show only the 3 upcoming exams; clicking 'Completed' shows only the 2 completed ones. Both the search and filter work together simultaneously.

Schedule Exam Modal

The '+ Schedule Exam' button opens an add exam modal with fields for subject, exam code, type, date, time, duration, room number, and expected student count. All required fields are validated before submission. The new exam is appended to the state array and appears in the table immediately. A two-step delete confirmation pattern is used for deletion — clicking the delete icon requires a second confirmation action before the record is removed, preventing accidental data loss.

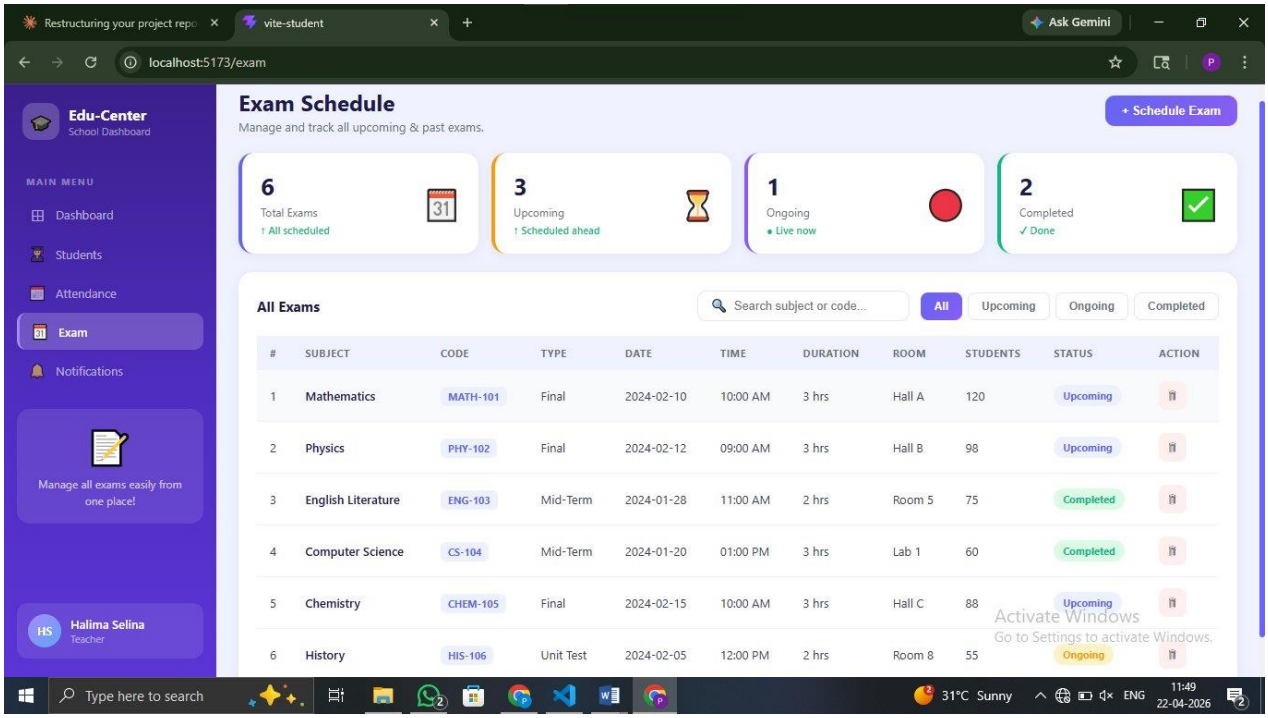

Figure 7 — Exam Module: Stat Cards, Exam Table with Codes, Status Badges, and Filters

5.5 Notifications Module

The Notifications module at /notifications serves as the central alert centre for the school. It displays all system-generated notifications categorised by type, with read/unread tracking and bulk management options.

Notification Summary Cards

Four cards at the top of the page show: Total notifications (12), Unread (4 — 'Needs attention' in orange), Read (8 — 'Already seen' in green), and Exam Alerts (3 — 'Exam related'). The Unread card uses an orange highlight to draw immediate attention to pending notifications, helping teachers prioritise what they need to review.

Notification Types and Categories

Each notification belongs to one of five categories: Exam, Attendance, Event, Finance, or General. Each type has its own icon, background colour, and badge label. For example, Exam notifications show a calendar icon with a blue badge, Attendance alerts show a grid icon with an orange badge, and Event notifications show a party icon with a green badge. These visual distinctions allow users to immediately identify the nature of a notification without reading the full text.

Filter and Search

A row of filter pills — All, Unread, Exam, Attendance, Event, Finance, General — allows users to view only notifications of a specific type. A search bar allows searching notification text content. Both work in

combination. For example, a teacher can click 'Attendance' and then type a student name to find specific attendance-related alerts.

Individual Notification Cards

Each notification is displayed as a card with: the type icon, the notification title in bold, the type badge, the full message body (e.g. 'Mathematics final exam has been rescheduled to Feb 15, Hall A at 10:00 AM'), a timestamp (e.g. '2 min ago'), a blue dot on unread notifications, and a dismiss (x) button on some cards. Unread notifications have a blue left-border accent to visually distinguish them from read ones.

Mark All Read and Clear All

Two action buttons in the top-right of the page — 'Mark all read' and 'Clear all' — allow bulk management. Clicking 'Mark all read' sets all notification read states to true, removing the blue unread dots and the count badge. 'Clear all' removes all notifications from the list, leaving a clean empty state with a contextual prompt.

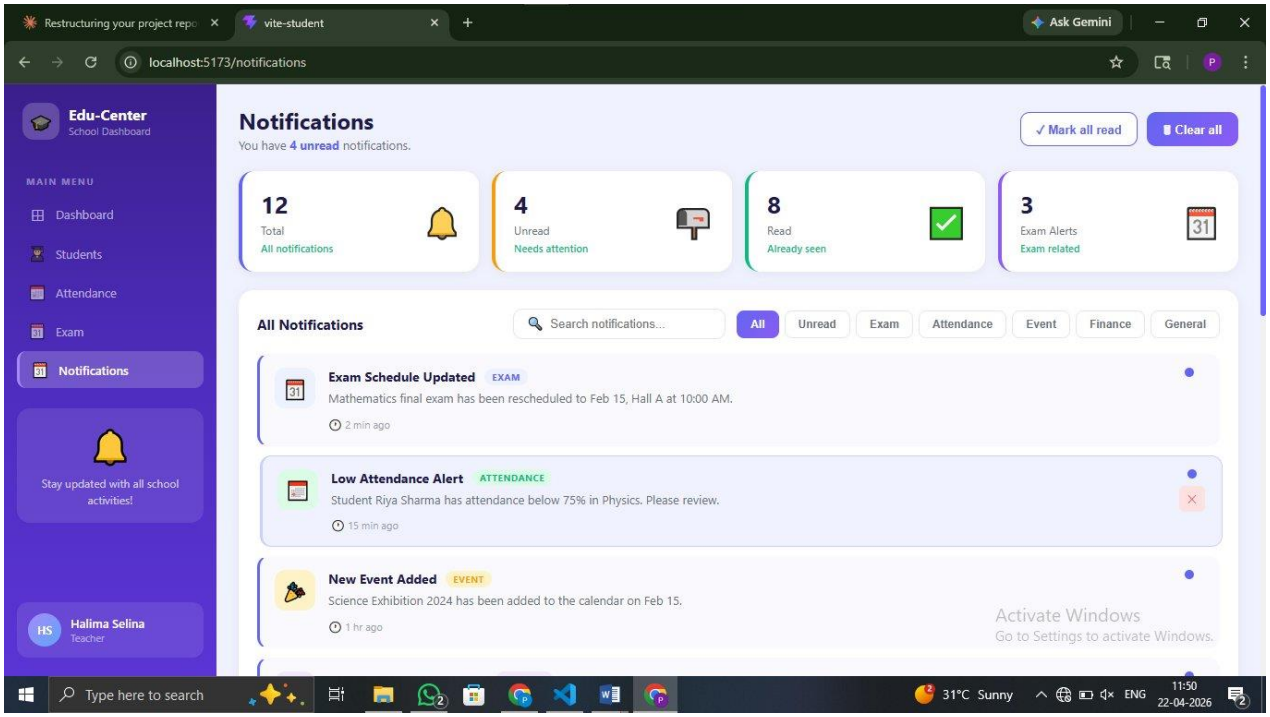

Figure 8 — Notifications: Summary Cards, Type Categories, Filter Pills, and Notification Cards

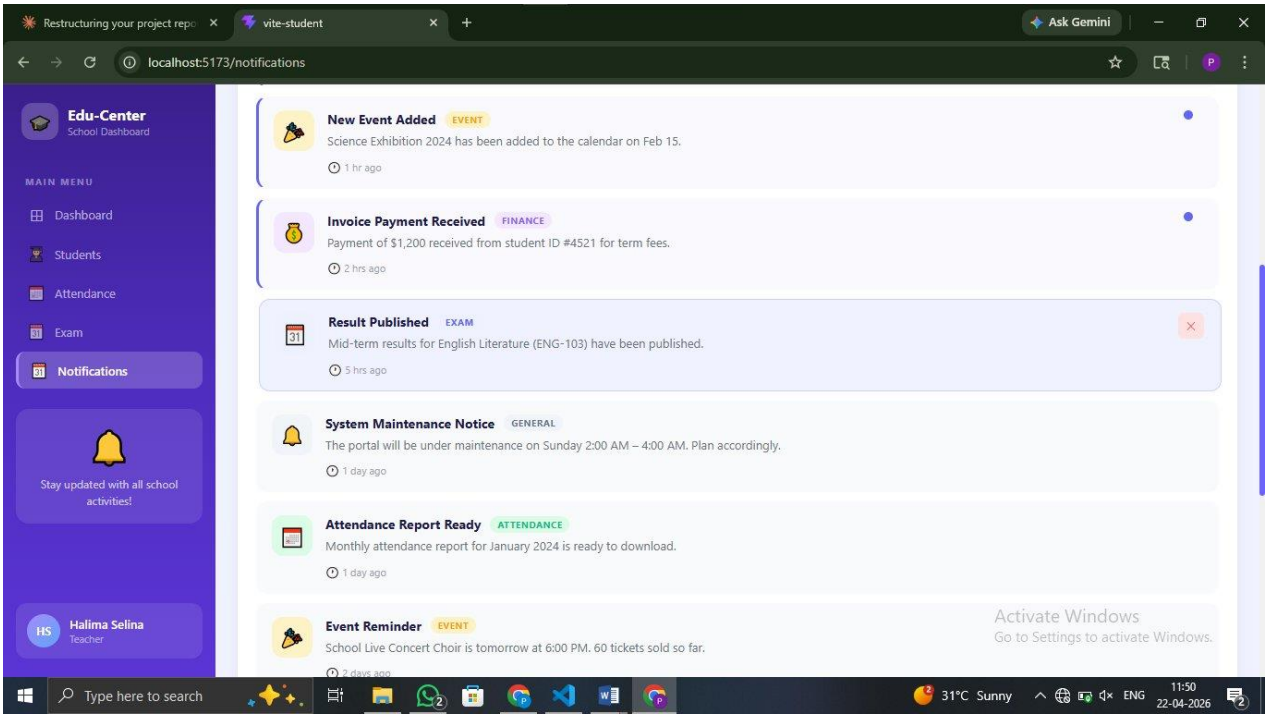

Figure 9 — Notifications: Finance, Exam, General, and Event Notification Types

6. Code Explanation

6.1 TypeScript Data Models

Every data entity in the application is defined as a TypeScript interface. For example, the Student interface includes fields typed as string (name, grade, subject, status), number (score), and a union type for status ('Active' | 'Inactive'). The Exam interface uses a union for its status field ('Upcoming' | 'Ongoing' | 'Completed'). The Notification interface uses a union type for category ('exam' | 'attendance' | 'event' | 'finance' | 'general'). Using union types instead of plain string means TypeScript will throw a compile-time error if any part of the code tries to set an invalid status value — catching whole categories of bugs before the app even runs.

6.2 CRUD Implementation

The student CRUD operations are all implemented as pure state transforms. Adding a student creates a new object with a generated ID and uses the functional form of setState to append it: setStudents(prev => [...prev, newStudent]). Editing uses .map() to replace the matching record: setStudents(prev => prev.map(s => s.id === editId ? updatedStudent : s)). Deleting uses .filter(): setStudents(prev => prev.filter(s => s.id !== deleteId)). None of these operations require any API call or external library — the UI updates instantly because React re-renders the component whenever state changes.

6.3 Real-Time Search and Filter

The search functionality chains .filter() and .toLowerCase() to match the search query against relevant fields. For example, in the Students module: const filtered = students.filter(s => s.name.toLowerCase().includes(query.toLowerCase()) || s.subject.toLowerCase().includes(query.toLowerCase())). This filtered array is then passed through the grade filter: if (selectedGrade !== 'All') filtered = filtered.filter(s => s.grade === selectedGrade). The final filtered array is what gets rendered in the table — no debouncing is needed at this data scale.

6.4 Pagination Logic

Pagination is implemented manually without any library. A currentPage state variable (starts at 1) and a ITEMS_PER_PAGE constant (8) are used to compute the visible slice: const paginated = filtered.slice((currentPage - 1) * ITEMS_PER_PAGE, currentPage * ITEMS_PER_PAGE). The total page count is Math.ceil(filtered.length / ITEMS_PER_PAGE). The Previous button is disabled when currentPage === 1 and the Next button when currentPage === totalPages. When the search query changes, currentPage resets to 1 to prevent the user from being stuck on an empty page.

6.5 SVG Charts

The donut chart in the Attendance module is a pure SVG element. A circle element is given a stroke-dasharray value calculated as: const circumference = 2 * Math.PI * radius; const presentDash = (presentPercent / 100) * circumference. Multiple overlapping circles, each rotated by their cumulative start angle using a stroke-dashoffset, create the multi-segment donut effect. The School Performance chart on the Dashboard uses SVG polyline elements with calculated point coordinates based on normalised data values against the chart height. A subtle gradient fill uses an SVG linearGradient definition. No chart library (Recharts, Chart.js, D3) is used anywhere.

6.6 Modal Pattern

Modals are implemented using a fixed-position overlay div that covers the entire screen. Clicking the overlay backdrop calls the close handler (setModalOpen(false)). Inside the modal, event.stopPropagation() is called on the modal container's onClick to prevent clicks inside the modal from bubbling up to the backdrop and accidentally closing it. Modal entrance animation uses a CSS keyframe: @keyframes slideUp with transform: translateY(20px) to translateY(0) and opacity 0 to 1 over 200ms.

7. Testing

Manual testing was conducted across all five modules after each feature was added. The following scenarios were tested and verified:

Test Scenario	Module	Result
Add a new student — fill all fields and submit	Students	PASS
Form validation — submit with empty required fields	Students	PASS
Real-time search — type name, table filters instantly	Students	PASS
Grade filter — select 10th, only 10th grade shown	Students	PASS
Edit student — pre-filled modal, save updates record	Students	PASS
Pagination — navigate pages, disable on boundaries	Students	PASS
Attendance quick-mark — P/L/A toggle, only one active	Attendance	PASS
Donut chart — percentages update with attendance data	Attendance	PASS
Schedule exam — add exam, appears in table	Exams	PASS
Exam status filter — All/Upcoming/Ongoing/Completed	Exams	PASS
Delete exam — two-step confirmation before removal	Exams	PASS
Mark all notifications as read — unread count goes to 0	Notifications	PASS
Notification type filter — shows only selected type	Notifications	PASS
Cross-page navigation — sidebar links, no page reload	All Modules	PASS
No console errors on hard reload	All Modules	PASS

8. Limitations

The following limitations exist in the current version (v1.0.0) and are acknowledged honestly:

- No back-end or database — all data is stored in React component state and resets on page refresh
- No user authentication or login — any user can access all modules without credentials
- No data persistence — localStorage integration was not added in this version
- Mobile responsiveness is partially implemented — the layout is optimised for desktop (1280px+) screens
- No automated unit tests (Jest / React Testing Library) — all testing was done manually
- No accessibility audit — ARIA labels and full keyboard navigation are not implemented

9. Skills Demonstrated

This project demonstrates the following technical competencies relevant to front-end developer roles:

- **React 18 Development** — Functional components, hooks (useState, useEffect), component-based architecture across 5 fully independent modules
- **TypeScript** — Strict typing throughout — interfaces for all data models, union types for status fields, zero use of 'any'
- **CRUD Operations** — Full Create, Read, Update, Delete for student records and exam entries using pure React state transforms
- **Data Visualisation** — Custom SVG donut chart, area chart, and bar chart built from scratch — no charting library
- **Real-Time Search** — Chained .filter() logic with grade filter and pagination reset — fast and responsive on every keystroke
- **React Router v6** — SPA routing with useNavigate for programmatic sidebar navigation — no full-page reloads
- **Pure CSS Design** — Complex layouts including sidebar gradient, card hover effects, modal animations, status badges — no UI framework
- **Form Validation** — Per-field inline error messages using controlled inputs and submit-time validation logic
- **TypeScript Architecture** — Typed interfaces, props, and state throughout; union types prevent invalid data at compile time
- **Problem Solving** — Pagination, modal click-through prevention, SVG chart calculations, multi-filter logic — all solved independently

10. Future Enhancements

The following enhancements are planned for future versions of the Student ERP System:

Backend Integration

The most impactful next step is connecting the front-end to a Node.js + Express REST API. The component architecture is already structured to make this transition clean — all data operations are isolated in event handlers that currently update local state, and these can be replaced with axios or fetch API calls. MongoDB would be used for the database layer, with Mongoose schemas mapping directly to the existing TypeScript interfaces.

User Authentication

Implementing JWT-based authentication with role-based access control (RBAC) would allow three user types: Admin (full access), Teacher (attendance and notifications only), and Student (read-only view of own records). React Router's protected route pattern would guard each module based on the logged-in user's role.

Mobile Responsiveness

A collapsible sidebar using a hamburger menu toggle, combined with CSS Grid media queries at 768px and 480px breakpoints, would make the dashboard fully usable on tablets and mobile phones. The table-heavy Student and Exam modules would switch to card-based layouts on small screens.

Data Export

Adding PDF and CSV export options using jsPDF and PapaParse would let administrators download student records, attendance reports, and exam schedules for offline use, printing, and record-keeping.

Automated Testing

Adding Jest and React Testing Library tests for the core CRUD operations, form validation logic, and pagination would significantly improve code reliability and provide a safety net for future refactoring.

11. How to Run the Project

11.1 Prerequisites

- Node.js v16 or above installed
- npm or yarn package manager
- Git installed and configured

11.2 Setup Instructions

Step	Command / Action
1.	git clone https://github.com/Pranali027-blip/student-management-system.git
2.	cd student-management-system
3.	npm install
4.	npm run dev
5.	Open browser at http://localhost:5173

12. Conclusion

The Edu-Center Student ERP System demonstrates strong front-end fundamentals through a production-ready application built with React 18, TypeScript, and modern CSS. Its clean architecture, performance-focused approach, and modular design make it easy to extend with backend integration and additional features. The project highlights practical problem-solving, attention to detail, and the ability to build scalable, maintainable user interfaces. Overall, it reflects readiness to contribute effectively in a professional front-end development environment.

13. References

- React 18 Official Documentation — react.dev
- TypeScript Handbook — typescriptlang.org/docs
- React Router v6 Documentation — reactrouter.com/en/main
- Vite Official Documentation — vitejs.dev
- SVG stroke-dasharray technique — css-tricks.com
- CSS Grid Complete Guide — css-tricks.com/snippets/css/complete-guide-grid
- GitHub Repository — github.com/Pranali027-blip/student-management-system